Document Number: P3248R4.
Date: 2025-06-19.
Reply to: Gonzalo Brito Gadeschi <gonzalob _at_ nvidia.com (http://nvidia.com)>.
Authors: Gonzalo Brito Gadeschi.
Audience: LWG.

Require `[u]intptr_t`

Changelog

- **R4**:
  - Cleanup instances in the standard library that depend on this change (e.g. that handle `[u]intptr_t` being optional).
  - LEWG Review:

    - ACTION ITEM: Search for instances in the library wording that depend on this change
    - POLL: Forward “P3248R3: Require [u]intptr_t” and produce a new revision with the action item above (to be checked by Mark de Wever and Detlef Vollmann) to LWG for C++29.

SF	F	N	A	SA
11	8	1	0	0

Outcome: strong consensus in favour

- **R3**:
    - SG22 / WG14 saw this, and have no compatibility concerns with C.
    - Updated Wording to account for potential future adoption of P3348 (Update C++ to C23).

- **R2**:
    - [SG1 Poll]:
      There are no SG1 concerns with P3248R1

SF	F	N	A	SA
6	5	0	0	0

- **R1**:
    - Add "Header file inconsistency between C and C++" discussion to "Design" section.
    - Add context of C programming language efforts to require `[u]intptr_t`.
    - Add clarifications with respect to Memory Tagging.
    - Add C23 specification of `[u]intptr_t`.
    - Add impact analysis on conforming and non-conforming implementations.

- **R0**: initial draft.

Motivation

Proposals like P2835 (https://wg21.link/p2835) and P3125 (https://wg21.link/P3125) use `[u]intptr_t` as an integer type capable of holding a pointer value in their APIs[1]. However, `[u]intptr_t` being *optional* forces sub-optimal design choices such as making APIs optional or introducing workarounds.

The potential absence of `[u]intptr_t` compromises the portability of high-level software and attempts to address this introduce software engineering overheads and potential portability bugs, as seen in libvlc PR#1519 (https://code.videolan.org/videolan/vlc/-/merge_requests/1519).

This proposal advocates for requiring `[u]intptr_t` in C++ to ensure that all C++ code can rely on integer types capable of holding a pointer value.

Status quo

C Programming language semantics of `[u]intptr_t`

The ISO/IEC 9899:2023 Working Draft (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3096.pdf) specifies `[u]intptr_t` semantics as follows:

> **7.22.1.4 Integer types capable of holding object pointers**
>
> 1. The following type designates a signed integer type, other than a bit-precise integer type, with the property that any valid pointer to `void` can be converted to this type, then converted back to pointer to `void`, and the result will compare equal to the original pointer
>
>   `intptr_t`
>
>   The following type designates an unsigned integer type, other than a bit-precise integer type, with the property that any valid pointer to `void` can be converted to this type, then converted back to pointer to `void`, and the result will compare equal to the original pointer:
>
>   `uintptr_t`
>
>   These types are optional.

Other sections of the specification provide additional operations that preserve `[u]intptr_t` values:

- `memcpy`
- I/O functions like `fprintf` / `fscanf` on `[u]intptr_t`.

ISO/IEC CD TS 6010 - A provenance-aware memory object model for C (https://www.iso.org/standard/81899.html) (N3005 (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3005.pdf)) explores extending these guarantees.

C++'s [expr.reinterpret.cast#5] (https://eel.is/c++draft/expr.reinterpret.cast#5) brings C `[u]intptr_t` semantics into C++ as follows:

> A value of integral type or enumeration type can be explicitly converted to a pointer. A pointer converted to an integer of sufficient size (if any such exists on the implementation) and back to the same pointer type will have its original value ([basic.compound]); mappings between pointers and integers are otherwise implementation-defined.

C++ [cstdio.syn#1] (https://eel.is/c++draft/cstdio.syn#1) imports `frpintf` / `fscanf` from C.

Requiring `[u]intptr_t` in the C Programming Language

The C programming language proposal N2889 (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2889.htm) explored requiring `[u]intptr_t`. It was rejected for C23 but adopted into ISO/IEC CD TS 6010 - A provenance-aware memory object model for C (https://www.iso.org/standard/81899.html) (N3005 (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n3005.pdf)) to enable C to gain experience with the proposal. There is consensus that this the right approach, but there is not enough implementation experience.

Impact analysis

Impact on conforming implementations

A survey found ubiquitous support for `[u]intptr_t` in *conforming* C++ implementations (*):

- C++ Standard Library implementations assume `[u]intptr_t` are available: libstdc++, libc++, and Microsoft STL.

- C++ Compilers supporting `[u]intptr_t` on all targets, including those with non-standard pointer sizes: GCC, Clang, MSVC.
- C++ Platform ABIs specify the size and alignment of pointers and the calling convention of Integer types, fixing the ABI of `[u]intptr_t`. *Extended integer types* avoid breaking the ABI of `intmax_t` when introducing a wider `[u]intptr_t` (this used to be a problem, see N2889 (https://www.open-std.org/jtc1/sc22/wg14/www/docs/n2889.htm)).

We did not find any *conforming* implementation that is inconsistent in C and C++ with respect to the availability of `[u]intptr_t`: all implementations found provide these types in the headers of both programming languages.

We did not find any *conforming* implementation that:

- would stop conforming if C++ were to require `[u]intptr_t`, or
- does not already provide `[u]intptr_t`.

Therefore, we conclude that C++ requiring `[u]intptr_t`:

- does not regress current implementation support, and
- does not require any implementation effort,

for any currently conforming implementation.

(*) many C++ implementations are not conforming in one way or antoher, but here we focus on pointers.

Impact on non-conforming implementations

Full support for `[u]intptr_t` cannot be expected on platforms that lack full support for pointers. All the non-conforming implementations found, are non-conforming with respect to pointer support. For example, their I/O functions (`fprintf` / `fscanf`) or `memcpy` to unaligned addresses do not uphold pointer round-trips (e.g. via `%p`) validity requirements.

We evaluate the impact on these implementations in terms of what "partial" support for `[u]intptr_t` can be provided and at what effort (e.g. at least to document which partial support, if any, is provided).

We found that the following non-conforming platforms would *not* be impacted by C++ requiring `[u]intptr_t`:

- **CHERI C++**: already provides `[u]intptr_t` documenting limitations on its support. For more details, see, e.g., the CHERI C/C++ Programming Guide (https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-947.pdf) or the more recent: *Zaliva et al.*, Formal Mechanised Semantics of CHERI C: Capabilities, Undefined Behaviour, and Provenance (https://www.cl.cam.ac.uk/~pes20/asplos24spring-paper110.pdf), ASPLOS '24.

We found that the following non-conforming platforms may be impacted by C++ requiring `[u]intptr_t`:

- **IBM i (https://www.ibm.com/products/ibm-i?utm_content=SRCWW&p1=Search&p4=43700074687253318&p5=e&p9=587000822100044 0&gclid=EAIaIQobChMI9p6L2tXBhgMVKhOtBh0RBApyEAAYASAAEgKFSvD_BwE&gclsrc=aw.ds)** (see also IBM AS/400 (https://en.wikipedia.org/wiki/IBM_AS/400)): uses PowerPC AS Tagged Memory Extensions (https://www.devever.net/~hl/ppcas). Its ILE C++ compiler already documents standards compliance limitations, including lack `[u]intptr_t` (even though these types are currently optional). Whether it can implement `[u]intptr_t` is to be determined, but if it can, whether it does so may depend on other factors like customer demand.
- **Elbrus (https://en.wikipedia.org/wiki/MCST)** has memory tagging (https://en.wikipedia.org/wiki/Tagged_architecture) capabilities (https://news.ycombinator.com/item?id=27494357): in protected mode, pointers are 128-bit wide and include a memory address, an object size, and an offset, but `[u]intptr_t` is only 64-bit wide and does not support ptr2int2ptr round-trips (https://github.com/ivmai/libatomic_ops/issues/61). CMake (https://cmake.org/cmake/help/latest/variable/CMAKE_LANG_COMPILER_ID.html) and libfmt (https://github.com/fmtlib/fmt/pull/3057) support its compiler, and the latter employs a fallback (https://github.com/fmtlib/fmt/blob/8ee6c9401471b402e2e33f4671edcf74af33e20c/include/fmt/format.h#L429) in case `[u]intptr_t` is not available. Whether it can implement `[u]intptr_t` is to be determined, and if it can, whether it does so may depend on other factors.

Header file inconsistency between C and C++

On a platform in which the C implementation does not provide this type (only non-conforming implementations found), the `<stdint.h>` header does not provide this type in the C programming language (e.g. when processed by a C compiler).

Per [support.c.header.other.1] (), in C++ `<stdint.h>` has the same content as `<cstdint>`. In a platform in which `[u]intptr_t` is not available to C via `<stdint.h>`, it is required to be available to C++ via both `<stdint.h>` and `<cstdint>`:

```
// foo.c - C compiler
#include <stdint.h> // C header processed by C compiler
intptr_t val;       // ill-formed

// bar.cpp - C++ compiler
#include <stdint.h>  // C header processed by C++ compiler
intptr_t val;        // well-formed

// baz.cpp - C++ compiler
#include <cstdint>  // C++ header processed by C++ compiler
std::intptr_t val;  // well-formed
```

This disconnect in platforms in which the C implementation does not provide `[u]intptr_t` may impact developer productivity in those platforms.

Design

Design alternatives:

1. C++ requires `[u]intptr_t`.
2. C++ adds new integer types - different from `[u]intptr_t` - capable of holding a pointer value.
3. Do nothing.

This proposal advocates for Option 1, i.e., for C++ to require `[u]intptr_t`, because:

- **Pre-existing code**: All implementations surveyed provide these on all platforms. This has led to a large corpus of pre-existing code using `[u]intptr_t`. Requiring `[u]intptr_t` makes this code portable to all platforms C++ supports. Inventing new C++ types would make this code non-idiomatic and cause significant churn on all ecosystems for little added value.
- **Compatibility with C**: By requiring these types in C++ with the same semantics as C we ensure C++ remains forward compatible with C eventually requiring these types, since if that were to happen, C++ would get their exact same semantics and ABI. This is particularly important with respect to the pointer provenance rules as specified in TS

6010 (https://www.iso.org/standard/81899.html). Adding new C++ types that are not available in C would reduce C++'s compatibility with C. There is however a nuanced header file inconsistency between C and C++ that is covered in the next section.

- **ABI**: Platforms whose ABI specifies `intmax_t` to be smaller than the platform's pointer size are allowed to provide wider `[u]intptr_t` integer types since C23 and C++23 due to extended integer type support.
- **Cost**: There is a cost to doing nothing. Significant time was spent on `atomic_ref::address` to find a sub-optimal solution when the right solution everyone agrees on is `uintptr_t`.

Usage Guideline

`[u]intptr_t` is well suited for C++ language or C++ Standard Library APIs that need an integer type capable of holding a *pointer value*, i.e., an integer type with a lossless conversion from/to pointer.

Some features or APIs may only need an integer type capable of holding a *pointer address*. C and C++ do not currently provide an integer type suited for this use case, but some implementations do provide it as an extension, in platforms were this distinction is crucial, e.g., CHERI C/C++ implementations provide `ptraddr_t` in `<stddef.h>` (the CHERI C/C++ Programming Guide (https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-947.pdf) is currently outdated and mentions `vaddr_t` instead of `ptraddr_t`).

Wording changes

Modify [cstdint.syn] (https://eel.is/c++draft/cstdint.syn#1):

1. The header `<cstdint>` supplies integer types having specified widths, and macros that specify limits of integer types.

```
// all freestanding
namespace std {
  using int8_t         = signed integer type;   // optional
  using int16_t        = signed integer type;   // optional
  using int32_t        = signed integer type;   // optional
  using int64_t        = signed integer type;   // optional
  using intN_t         = see below;             // optional

  using int_fast8_t    = signed integer type;
  using int_fast16_t   = signed integer type;
  using int_fast32_t   = signed integer type;
  using int_fast64_t   = signed integer type;
  using int_fastN_t    = see below;             // optional

  using int_least8_t   = signed integer type;
  using int_least16_t  = signed integer type;
  using int_least32_t  = signed integer type;
  using int_least64_t  = signed integer type;
  using int_leastN_t   = see below;             // optional

  using intmax_t       = signed integer type;
  using intptr_t       = signed integer type;   // optional

  using uint8_t        = unsigned integer type; // optional
  using uint16_t       = unsigned integer type; // optional
  using uint32_t       = unsigned integer type; // optional
  using uint64_t       = unsigned integer type; // optional
  using uintN_t        = see below;             // optional

  using uint_fast8_t   = unsigned integer type;
  using uint_fast16_t  = unsigned integer type;
  using uint_fast32_t  = unsigned integer type;
  using uint_fast64_t  = unsigned integer type;
  using uint_fastN_t   = see below;             // optional

  using uint_least8_t  = unsigned integer type;
  using uint_least16_t = unsigned integer type;
  using uint_least32_t = unsigned integer type;
  using uint_least64_t = unsigned integer type;
  using uint_leastN_t  = see below;             // optional

  using uintmax_t      = unsigned integer type;
  using uintptr_t      = unsigned integer type; // optional
}

#define INTN_MIN         see below
#define INTN_MAX         see below
#define UINTN_MAX        see below
```

```
#define INT_FASTN_MIN    see below
#define INT_FASTN_MAX    see below
#define UINT_FASTN_MAX   see below

#define INT_LEASTN_MIN   see below
#define INT_LEASTN_MAX   see below
#define UINT_LEASTN_MAX  see below

#define INTMAX_MIN       see below
#define INTMAX_MAX       see below
#define UINTMAX_MAX      see below

#define INTPTR_MIN       see below              // optional
#define INTPTR_MAX       see below              // optional
#define UINTPTR_MAX      see below              // optional

#define PTRDIFF_MIN      see below
#define PTRDIFF_MAX      see below
#define SIZE_MAX         see below

#define SIG_ATOMIC_MIN   see below
#define SIG_ATOMIC_MAX   see below

#define WCHAR_MIN        see below
#define WCHAR_MAX        see below

#define WINT_MIN         see below
#define WINT_MAX         see below

#define INTN_C(value)    see below
#define UINTN_C(value)   see below
#define INTMAX_C(value)  see below
#define UINTMAX_C(value) see below
```

2. The header defines all types and macros the same as the C standard library header `<stdint.h>` except that the types `intptr_t` and `uintptr_t` and the macros `INTPTR_MIN`, `INTPTR_MAX`, and `UINTPTR_MAX` are always defined and are not optional. See also: ISO/IEC 9899:2018, 7.20.

3. All types that use the placeholder N are optional when N is not 8, 16, 32, or 64. The exact-width types `intN_t` and `uintN_t` for N = 8, 16, 32, and 64 are also optional; however, if an implementation defines integer types with the corresponding width and no padding bits, it defines the corresponding typedef-names. Each of the macros listed in this subclause is defined if and only if the implementation defines the corresponding

typedef-name.

[Note 1: The macros INTN_C and UINTN_C correspond to the typedef-names `int_leastN_t` and `uint_leastN_t`, respectively. — end note]

Modify `<cstdint>`:

```
#define INTPTR_WIDTH see below ~~// optional~~
#define UINTPTR_WIDTH see below ~~// optional~~
```

Drafting note: no feature test macro required per LEWG.

Modify [tab:format.type.ptr] (https://eel.is/c++draft/tab:format.type.ptr)

~~If uintptr_t is defined,~~
to_chars(first, last, reinterpret_cast<uintptr_t>(value), 16)
with the prefix 0x inserted immediately before the output of to_chars~~; otherwise, implementation-defined.~~

Modify [atomics.alias] (https://eel.is/c++draft/atomics.alias):

The type aliases atomic_intN_t, and atomic_uintN_t~~, atomic_intptr_t, and atomic_uintptr_t~~ are defined if and only if intN_t and uintN_t~~, intptr_t, and uintptr_t~~ are defined, respectively.

Acknowledgements

Jens Gustedt for their help with coordinating with WG14, TS 6010, N2889, and establishing a contact with the IBM AS/400 team. Nikolaos Strimpas and and Alibek Omarov for their help in documenting the impact to Elbrus. Aaron Ballman, Jessica Clarke, Jonathan Wakely, Ville Voutilainen, and many others, for feedback that resulted in substantial improvements to the proposal.

1. This does not imply that these proposals make correct use of these types; the Usage Guideline (https://hackmd.io/U-X9IVCjSEqjXDcw9AjlnQ#Usage-Guideline) section covers that. ↩